

Test-Time Scaling and GRPO for Reasoning LLMs

Yafu Li

The Chinese University of Hong Kong
Shanghai AI Laboratory

CUHK Lecture

This slide deck is vibe-built with Codex.

Acknowledgements

Many of the slides in this lecture are adapted from the sources below. Copyrights belong to the original authors.

- Prof. Wing C. Lau, *Teaching LLMs to Reason and Reasoning LLMs*, CUHK course lecture deck
- Lilian Weng, *Why We Think*, Lil'Log, May 2025
- Ben Prystawski, Michael Y. Li, Noah D. Goodman, *Why think step by step? Reasoning emerges from the Locality of Experience*, NeurIPS 2023
- Jason Wei et al., *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, NeurIPS 2022
- Xuezhi Wang et al., *Self-Consistency Improves Chain of Thought Reasoning in Language Models*, ICLR 2023
- Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, NeurIPS 2023
- Charlie Snell et al., *Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters*, 2024
- Bradley Brown et al., *Large Language Monkeys: Scaling Inference Compute with Repeated Sampling*, 2024
- Yafu Li et al., *Test-Time Preference Optimization: On-the-Fly Alignment via Iterative Textual Feedback*, ICML 2025
- Niklas Muennighoff et al., *s1: Simple test-time scaling*, 2025
- John Schulman et al., *Proximal Policy Optimization Algorithms*, 2017
- Zhihong Shao et al., *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*, 2024
- DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025
- Yafu Li et al., *LUFFY: Learning to Reason Under Off-Policy Guidance*, NeurIPS 2025

Yafu Li

Current Roles

- Postdoctoral Researcher, The Chinese University of Hong Kong
- Researcher, Shanghai AI Laboratory

Research Interests

- Scalable reasoning and alignment for large language models
- RLVR, off-policy training, inference-time optimization, and evaluation

Background

- Ph.D. in Computer Science, Zhejiang University and Westlake University
- Previous experience at Tencent AI Lab and Huawei Noah's Ark Lab

Selected Work

- *Test-Time Preference Optimization* (ICML 2025)
- *Learning to Reason under Off-Policy Guidance* (NeurIPS 2025)

Introduction

Large language models are increasingly strong on mathematics, coding, and other multi-step tasks, but strong reasoning does not come only from larger parameter counts or more pretraining.

Recent progress shows that performance can improve substantially by:

- allocating more compute at inference time,
- exploring longer or multiple reasoning paths,
- training the model to use that compute more effectively.

This lecture focuses on two closely related ideas:

- **Test-time scaling**: longer traces, multiple samples, revision, and search.
- **GRPO**: training reasoning behavior by comparing groups of sampled responses and optimizing toward better trajectories.

Main questions:

- Why does more inference-time compute often improve reasoning?
- How does GRPO connect train-time optimization with test-time exploration?

① Why we reason

- Why one-shot next-token prediction is insufficient for hard tasks
- Why intermediate reasoning improves locality, robustness, and adaptive compute
- What reasoning buys us in practice, and what we should be careful not to overclaim

② How we reason via test-time scaling

- Budgeted inference: width vs. depth as the two main scaling axes
- Repeated sampling, verifier-guided search, and sequential revision
- Hybrid strategies: TPO and the simple SFT-plus-budget-forcing recipe in s1

③ Boosting reasoning with GRPO

- From SFT to RL: policy gradients, PPO, and why GRPO simplifies reasoning RL
- RLVR with GRPO: DeepSeekMath and DeepSeek-R1/R1-Zero
- Beyond on-policy RL: off-policy guidance with LUFFY

Why do we want LLMs to reason?

This section answers four questions in order:

- **What is the problem?** Why is one-shot next-token prediction insufficient?
- **Why does reasoning help?** Why do intermediate steps improve inference?
- **Why do we care in practice?** What does reasoning buy us on real tasks?
- **What are the boundaries?** What should we be careful not to overclaim?

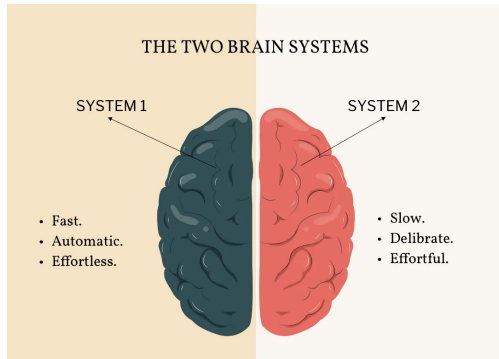
Fast and Slow Thinking

Human problem solving is often described through two modes:

- **System 1:** fast, automatic, intuitive, low effort.
- **System 2:** slower, deliberate, logical, effortful.

Many early LLM behaviors look much closer to **System 1**:

- produce the most likely continuation quickly,
- rely on local pattern completion,
- struggle when the task requires explicit multistep computation.



Why Intuition Alone Fails

One-shot generation is often good enough for:

- factual recall,
- fluent paraphrasing,
- short conversational continuations.

It is often **not** good enough for:

- arithmetic and symbolic manipulation,
- multi-step logical inference,
- tasks where early mistakes compound.

Example. Consider a GSM8K-style question:

“Tom has 3 bags. Each bag has 4 apples. He gives away 5 apples. How many apples are left?”

A direct answer token like 7 or 9 can be confused by shallow pattern matching.

But the chain-of-thought is simple:

$$3 \times 4 = 12, \quad 12 - 5 = 7$$

The final answer is easy *once the hidden subproblem is exposed*: first compute the total apples, then subtract what was given away.

This is the power of CoT: instead of memorizing the whole mapping from question to answer, the model can follow a sequence of **small, reliable intermediate steps**.

What Do We Mean by Reasoning?

In the LLM setting, reasoning usually means:

- solving questions that require **multiple intermediate steps**,
- decomposing a hard task into simpler subproblems,
- possibly using recursion, iteration, planning, or tools,
- then combining these partial results into a final answer.

This definition is broader than “showing a rationale to the user”.

A model may reason internally by:

- keeping track of state,
- exploring multiple options,
- revising mistaken partial solutions,
- optionally invoking external computation.

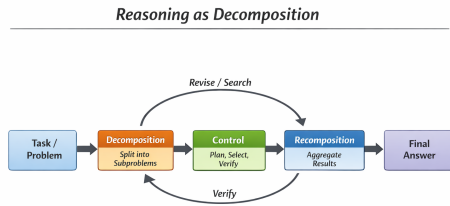
Reasoning as Decomposition

Reasoning = decomposition + control + recomposition

- **Decomposition:** split the task into manageable pieces.
- **Control:** decide what to do next, what to check, and when to branch.
- **Recomposition:** aggregate intermediate results into a final answer.

This is why reasoning is closely related to:

- planning and search,
- verification and revision,
- tool use when memory or arithmetic is not enough.



Reasoning = Decomposition + Control + Recomposition

Why Intermediate Variables Help

Intermediate steps are useful because many hard tasks are difficult to predict **directly**.

- Direct answer prediction asks the model to jump from problem to solution in one move.
- Intermediate reasoning introduces latent sub-results that are easier to estimate.
- These sub-results often correspond to local dependencies the model has seen more often in training.

In other words, chain-of-thought is not magic:

- it changes the statistical structure of the task,
- making inference easier by inserting useful hidden variables.

Hard jump: $p(y | x)$

Expose z :
$$p(y | x) = \sum_z p(y | z, x) p(z | x)$$

$$x \longrightarrow z_1 \longrightarrow z_2 \longrightarrow y$$

$$p(y | x) = \sum_{z_1, z_2} p(y | z_2, x) p(z_2 | z_1, x) p(z_1 | x)$$

Hidden variables can be subtotals, carries, subgoals, constraints, or checked partial results. Each local transition is easier than the full jump to the final answer.

Locality of Experience Hypothesis

Prystawski, Li, and Goodman propose a concrete explanation for why step-by-step reasoning helps:

- Direct prediction can fail because the relevant input-output pairs are rarely seen together.
- Reasoning helps when it replaces one weak long-range dependency with several strong local ones.
- Each intermediate step can be supported by many more examples from training.

This view explains why:

- reasoning can be more data efficient,
- intermediate variables improve hard inference,
- longer reasoning helps most when direct mapping is weak.

Example: multi-digit addition

Direct one-shot prediction:

$$p(95 \mid 37 + 58)$$

But the exact pair $(37 + 58, 95)$ may appear rarely.

Introduce local hidden variables:

$$z_1 = 7 + 8 = 15, \quad z_2 = \text{carry } 1$$

$$z_3 = 3 + 5 + 1 = 9, \quad y = 95$$

$$p(y \mid x) = \sum_{z_1, z_2, z_3} p(y \mid z_3, x) p(z_3 \mid z_2, x) \\ \cdot p(z_2 \mid z_1, x) p(z_1 \mid x)$$

The exact pair “ $37 + 58 = 95$ ” may be rare, but local facts like “ $7 + 8 = 15$ ” and “ $3 + 5 + 1 = 9$ ” are common.

Why Reasoning Helps Few-Shot Learning

Reasoning improves few-shot learning because demonstrations can teach **procedure**, not only answers.

- A short rationale can reveal how to transform input to output.
- One or a few examples may be enough to induce the correct multistep program.
- This is why chain-of-thought demonstrations can be much more sample efficient than direct answer exemplars.

The model is no longer imitating only the final output distribution. It is also imitating the **computational path** that generated that output.

Why Reasoning Helps Generalization

Reasoning is also attractive because it can improve **out-of-distribution robustness**.

- Memorized patterns fail when a problem differs from training examples on the surface.
- Decomposition can expose deeper reusable structure.
- Multistep reasoning can support transfer to new compositions of familiar local rules.

This does not guarantee full generalization, but it gives the model a better chance to:

- handle novel combinations,
- recover from superficial mismatches,
- build answers from reusable subskills.

Why Reasoning Helps Explainability and Diagnosis

Intermediate reasoning traces are valuable even when they are imperfect.

- They expose where the model's solution path went wrong.
- They provide a natural interface for verifier models, humans, or tools.
- They make error categories easier to localize than a wrong final answer alone.

For research and deployment, this matters because:

- debugging becomes easier,
- reward signals can target process quality,
- monitoring becomes more informative than only checking final answers.

Example: diagnosing a wrong answer

Problem: A box has 12 red balls and 8 blue balls. 5 red balls are removed. How many balls remain?

Model trace:

- 1 Total balls initially: $12 + 8 = 20$.
- 2 Remove 5 red balls: $20 - 5 = 16$.
- 3 Final answer: **16**.

Suppose the model instead writes:

- 1 Total balls initially: $12 + 8 = 20$.
- 2 Remove 5 red balls: $20 - 5 = 14$.
- 3 Final answer: **14**.

Without the trace, we only know the answer is wrong. With the trace, we can localize the failure to step 2.

Thinking in Tokens

In language models, extra reasoning is often implemented simply as **more generated tokens before the answer**.

- Scratchpads and CoT convert hidden computation into a token sequence.
- These tokens serve as temporary memory, partial derivations, or explicit subgoals.
- This makes reasoning easy to prototype because it reuses the autoregressive interface.

The important shift is conceptual:

- output tokens are no longer just the final response;
- they also become a **computational workspace**.

Concrete example from practice

A typical case: on olympiad-level math, a capable reasoning LLM spend **more than 60k tokens** on average before producing a clean final solution.

Rough breakdown:

- setup: ~1k–2k,
- trying ideas and dead ends: ~15k–20k,
- lemmas, case splits, revision: ~25k–30k,
- checking edge cases: ~10k–15k,
- final answer: often only a few hundred tokens.

Short answer, very long computation.

A Real Olympiad Example

The file `references/amo_example` gives a concrete instance of this phenomenon.

Problem type

- IMO-style combinatorial geometry on a triangular lattice.
- Task: among 15 lattice points in a subdivided equilateral triangle, find the smallest n such that any n chosen points must contain an isosceles triangle.

Why it is hard

- requires setting up the lattice coordinates,
- analyzing distances and symmetries,
- exploring constructions and counterexamples,
- reasoning about maximal isosceles-free subsets.

What the recorded trace looks like

- The stored response is about **101k tokens** long.
- Most of it is not the final answer, but exploratory computation: coordinate choices, distance formulas, failed constructions, and repeated checking.
- The final output is tiny by comparison:

6

This is the key point of test-time scaling: the visible answer can be short, while the reasoning workspace behind it is extremely long.

Reasoning Gives Us Adaptive Compute

Test-time reasoning introduces a new knob: **how much computation to spend on each problem**. Easy questions may need almost no deliberation, while hard questions may benefit from much longer traces. Unlike parameter scaling, this compute budget can be adjusted **online** at deployment time.

Example 1: easy task, little or no thinking

For simple queries, we may deliberately **turn off thinking mode**.

- translation or rewriting,
- quick factual lookup.

Extra reasoning mostly adds latency.

Example 2: hard task, force more thinking

For difficult problems, we can encourage longer reasoning by prompting the model to slow down.

- “wait”,
- “think step by step”,
- “double-check before answering”.

On hard math or coding, such cues can elicit longer exploration and verification.

Reasoning Matters Most on Hard Tasks

The benefit of reasoning is not uniform.

- On easy tasks, short direct decoding may already be enough.
- On nontrivial reasoning tasks, thinking time matters much more.
- Larger models often benefit more because they can use the extra compute more effectively.

Empirically, the largest gains tend to appear on:

- mathematics,
- coding,
- logic, planning, and other tasks with verifiable intermediate structure.

Reasoning Is Also a Training Signal

Once intermediate traces exist, they can supervise learning.

- We can fine-tune on good rationales.
- We can filter traces by correctness of the final answer.
- We can reward trajectories that lead to verifiable success.

This is a major reason reasoning matters for modern LLM training:

- it is useful at inference time (e.g., test-time scaling),
- and it also creates better optimization targets for post-training (e.g., SFT, RL).

Caveat: A Visible Trace Is Not Proof of True Reasoning

We should be careful not to overclaim.

- A model can produce a convincing rationale that is post hoc or partially unfaithful.
- Some gains may come from exposing more context rather than human-like reasoning.
- Visible CoT is useful, but it is not itself proof that the model reasons exactly as humans do.

So the right stance is pragmatic:

- reasoning traces are valuable as computation, supervision, and diagnostics;
- their faithfulness and interpretability still require careful evaluation.

Note 1

A **correct final answer** does not imply that the reasoning trace was sensible or faithful.

The model may:

- guess correctly,
- rationalize after the fact,
- mix correct and incorrect steps but still land on

Note 2

A **sensible reasoning process** does not always lead to the expected final answer.

The model may:

- make a small arithmetic slip at the end,
- stop too early,
- misread one condition despite an otherwise good

Limits and Open Questions

Once we know *why* reasoning helps, there are two kinds of questions.

What this lecture will address

- How to spend more inference-time compute: longer traces, multiple samples, search, and revision.
- How to train models to use that compute better: from SFT and RL basics to GRPO.
- How GRPO connects train-time optimization with test-time reasoning behavior.

This lecture focuses on the first set and only touches the second set as open problems.

What remains open / future interest

- How faithful reasoning traces really are.
- How to choose the right compute budget, stopping rule, or tool-use policy.
- How to monitor reward hacking and unfaithful process supervision.
- How to distill expensive reasoning into cheaper models without losing capability.

Takeaways: Why We Reason

- **Why one-shot prediction fails:** hard math, coding, and planning problems often require memory, decomposition, checking, and revision.
- **Why intermediate steps help:** chain-of-thought introduces useful hidden variables and converts weak long-range prediction into easier local transitions.
- **Why it matters in practice:** reasoning gives us adaptive compute, better diagnostics, stronger few-shot transfer, and better training signals.
- **What not to overclaim:** a visible trace is useful, but it is not automatic proof of faithful or human-like reasoning.
- **Main message:** reasoning is valuable because it improves how LLMs compute, not because it perfectly imitates how humans think.

Next: If reasoning helps, how do we scale it at inference time through longer traces, multiple samples, search, and revision?

If reasoning matters, how do we buy more of it at inference time?

This section follows a simple logic:

- **Allocate more compute** at test time instead of taking only one shot.
- **Spend it along two axes:** wider exploration and deeper refinement.
- **Choose the allocation carefully** because different problems prefer different strategies.
- **Train the model to use extra compute well** so additional thinking is actually productive.

From One-Shot Decoding to Budgeted Reasoning

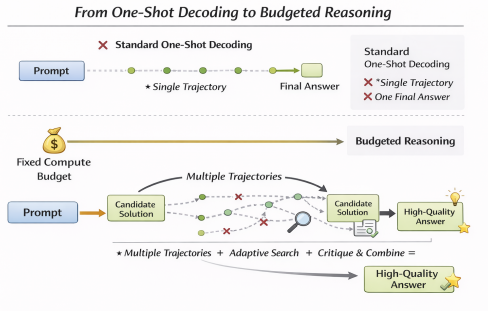
Standard decoding treats inference as one prompt, one trajectory, and one final answer.

Test-time scaling asks a different question:

“Given a fixed compute budget, how should we spend it to maximize answer quality?”

That budget can be used to:

- sample multiple candidate solutions,
- search and prune intermediate states,
- critique, revise, and combine trajectories adaptively.



Two Axes: Width and Depth

A useful organizing view is that test-time scaling has two main axes.

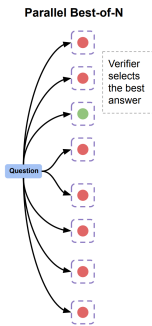
Width: explore more possibilities

- sample multiple independent reasoning paths,
- vote over or verify candidates,
- search a tree of partial solutions.

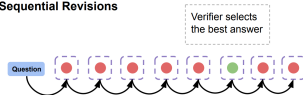
Depth: improve a path over time

- critique a draft,
- revise it sequentially,
- delay commitment and keep thinking.

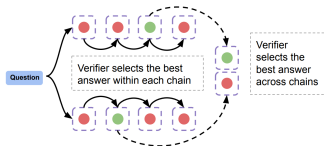
Most modern methods can be understood as choosing a point in this width–depth design space.



Sequential Revisions



Combining Sequential / Parallel



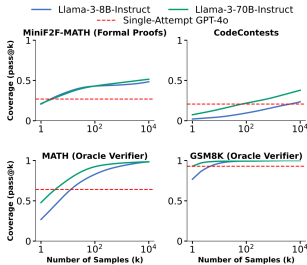
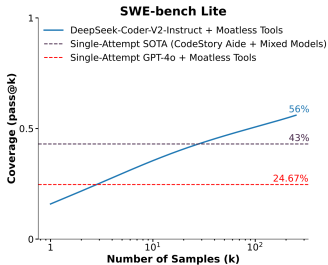
Width Scaling: Repeated Sampling

The simplest width strategy is repeated sampling:

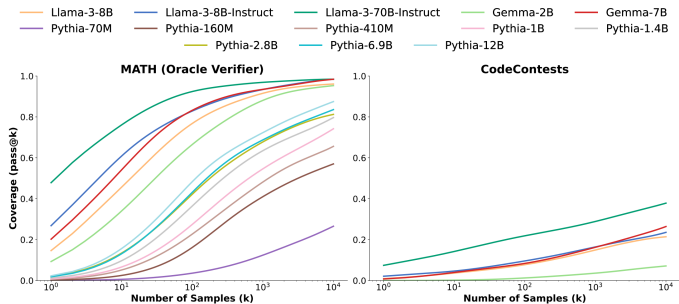
- generate many independent candidate solutions,
- keep temperature positive so trajectories differ,
- then select one final answer.

Large Language Monkeys shows that this simple strategy already scales surprisingly far:

- coverage improves as the number of samples grows,
- the gains persist across math, coding, proofs, and SWE-bench-style tasks.



What These pass@k Curves Actually Show



First: what is pass@k?

It is the probability that, among k sampled attempts, at least one is correct.

1. Smaller models can gain sharply from repeated sampling.

Coverage improves for almost every model on MATH, often with especially steep gains for smaller models.

2. **Pythia-160M** is a striking example.

On MATH, **Pythia-160M** rises from pass@1 = 0.27% to pass@10k = 57%.

3. The task still matters.

On CodeContests, Llama and Gemma improve, but the Pythia family stays at zero coverage even with 10,000 samples.

One plausible explanation is that Pythia saw less code-specific training data, so width cannot rescue missing capability by itself.

Repeated Sampling Depends on Coverage and Precision

Repeated sampling works only if two conditions hold:

- **Coverage:** among many samples, at least one candidate solves the problem.
- **Precision:** we can reliably identify which candidate is correct.

This distinction is important:

- automatic verifiers make precision easy in code (e.g., LeetCode) and formal proof (e.g., Lean),
- precision is harder in open-ended math or natural language settings,
- so more samples alone do not guarantee better final answers.

In practice, repeated sampling is strongest when the task provides a clean verification signal.

Best-of- N , Self-Consistency, and Selection

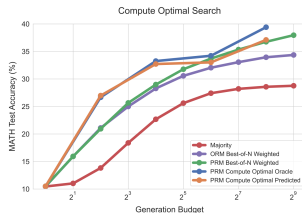
Once we have many samples, several selection rules are possible:

- **Majority voting / self-consistency:** choose the most common final answer.
- **Best-of- N :** score full solutions and keep the best one.
- **Verifier-based reranking:** select the candidate with the strongest outcome or process score.

These methods differ mainly in how much they trust:

- answer agreement,
- final-output quality,
- or intermediate reasoning quality.

Width helps generate diversity, but the selection rule determines whether that diversity converts into accuracy.



Width Alone Is Not the Whole Story

Pure repeated sampling spends compute only at the *trajectory level*:

- it evaluates complete answers after they are generated,
- but it does not guide the model while the reasoning is being constructed.

This creates two inefficiencies:

- many samples may share the same early mistake,
- compute may be wasted finishing low-potential trajectories.

The natural next step is to move from **sampling completed answers** to **searching over partial solutions**.

Search with Process Verifiers

Scaling LLM Test-Time Compute Optimally reframes test-time scaling with a proposer–verifier view:

- the **proposer** generates candidate reasoning steps,
- the **verifier** evaluates which steps or solutions look promising.

A strong process reward model (PRM) scores *intermediate steps*, not only the final answer.

That enables search procedures such as:

- beam search over partial traces,
- lookahead search with limited rollouts,
- weighted aggregation across candidate branches.

Beam Search vs. Best-of- N

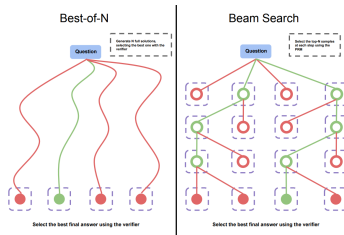
Search changes how compute is spent:

- **Best-of- N** explores many complete solutions independently.
- **Beam search** prunes weak branches early and keeps promising partial traces alive.

The empirical result in Snell et al. is nuanced:

- at smaller budgets, beam search often beats naive best-of- N ,
- at larger budgets, the gains can diminish,
- aggressive search can over-optimize against an imperfect verifier.

So stronger optimization is not automatically better optimization.



Difficulty Matters: Compute-Optimal Search

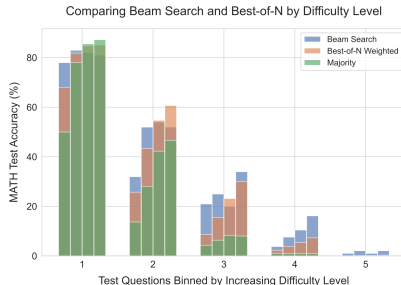
The main takeaway from Snell et al. is not just that search helps, but that **the best strategy depends on question difficulty**.

A useful intuition is:

- **Easier questions:** best-of- N may be enough; extra search can overfit verifier noise.
- **Harder questions:** beam search can help because the model must explore higher-level strategy choices.

This motivates a compute-optimal policy:

- estimate difficulty,
- allocate budget adaptively,
- and choose the search style that matches the regime.



Depth Scaling: Sequential Revision

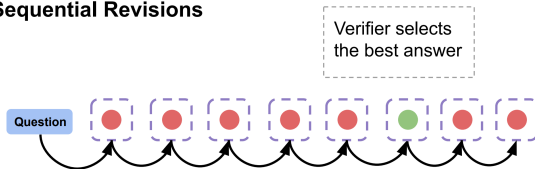
Width is not the only way to spend extra compute. We can also improve a *single path* over time.

Sequential revision works by:

- generating an initial answer,
- critiquing or reconsidering it,
- producing a revised answer conditioned on the previous attempt,
- repeating this process for several rounds.

In the proposer–verifier perspective, revision modifies the **proposal distribution** itself rather than only reranking finished outputs.

Sequential Revisions



Why Revisions Help, and When They Fail

Sequential revision is especially appealing when the first answer is *almost right*.

It can help because:

- later steps reuse earlier progress,
- local mistakes can be corrected without restarting from scratch,
- the model can double-check calculations or logic before finalizing.

But it can also fail:

- the model may keep polishing a fundamentally wrong plan,
- revisions can amplify spurious correlations,
- poor self-critique may create longer but not better reasoning traces.

Sequential vs. Parallel Compute

Snell et al. study the trade-off between:

- **parallel compute:** many independent samples,
- **sequential compute:** more revisions on each sample.

The pattern is intuitive and important:

- easier problems often benefit more from sequential refinement,
- harder problems need some parallel exploration of distinct strategies,
- the best ratio shifts with both difficulty and budget.

This is the depth analogue of compute-optimal search: not just *how much* compute, but *what mix* of compute.

A Unified View: Width and Depth Are Complementary

At this point, a clean picture emerges:

- **Width** improves the chance of finding a promising trajectory.
- **Depth** improves the chance of repairing and refining a promising trajectory.

Pure width wastes effort if promising drafts are not improved.

Pure depth wastes effort if the initial plan is fundamentally wrong.

Therefore, strong test-time scaling methods often combine:

- exploration across candidates,
- local optimization within candidates,
- and some adaptive rule for trading the two off.

An Extreme TTS Example: High-Compute Search

DeepSeekMath-V2 shows what test-time scaling looks like in the extreme regime for hard olympiad-style proof problems.

For one problem, the search loop scales **all three kinds of compute at once**:

- start with 64 proof samples,
- run 64 verification analyses for each proof,
- then refine for up to 16 iterations,
- with each model call capped at 128k tokens.

In each refinement iteration:

- keep the top 64 proofs,
- pair each with 8 analyses,
- generate one revised proof per pair,
- and verify the new proofs again.

So the worst-case number of model calls is

$$64 + 64 \times 64 + 16(64 \times 8 + 64 \times 8 \times 64) = 536,640.$$

If we upper-bound each call by 128k tokens, then

$$\text{tokens} \leq 536,640 \times 128,000 = 68,689,920,000 \approx 6.87 \times 10^{10}.$$

That is about **68.7 billion tokens for one problem** in the worst case.

The point is conceptual: at very high compute, TTS is no longer just “sample more” or “revise more”. It becomes a closed-loop search system over **candidates, critiques, and refinements**.

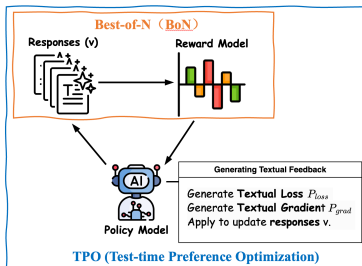
TPO: Test-Time Preference Optimization

Test-Time Preference Optimization (TPO) makes this hybrid idea explicit.

The basic loop is:

- sample multiple responses,
- score them with a reward model,
- convert reward differences into textual critiques,
- use those critiques to generate improved responses in the next iteration.

Unlike RLHF or DPO, TPO does not update model weights during inference. It performs *on-the-fly optimization in text space*.



TPO Combines Width and Depth

TPO can be read almost directly in the width–depth language:

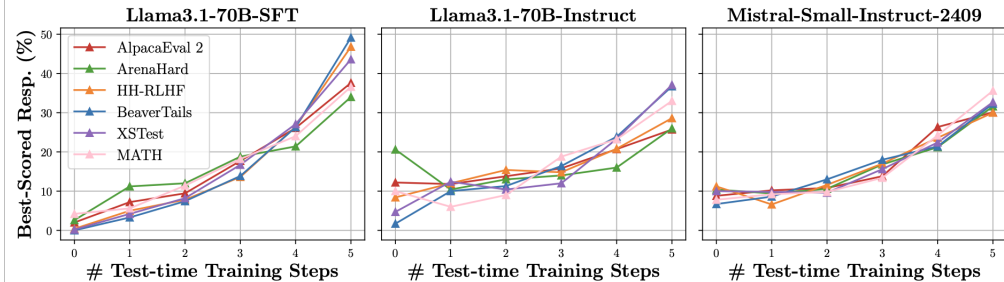
- **Width:** sample more candidates per iteration.
- **Depth:** run more rounds of critique and refinement.

This makes TPO a synthesis of:

- parallel sampling, which broadens search,
- sequential revision, which sharpens each candidate,
- reward-guided selection, which stabilizes the loop.

Conceptually, it turns numerical reward feedback into an explicit textual optimization process the model can follow step by step.

What TPO Adds to the Test-Time Scaling Story



Relative to pure best-of- N , the extra depth lets the system use feedback to move responses toward higher-reward regions instead of merely selecting among fixed candidates.

This figure shows **when** the best response appears during TPO iterations:

- for less aligned models, the best response often appears at later TPO steps,
- this highlights the value of **sequential revision** rather than relying only on initial sample quality,
- more aligned instruction-tuned models tend to produce strong responses earlier, so they need fewer corrective rounds.

Design Principles for Test-Time Scaling

Across these papers, a practical design checklist emerges:

- **Decide what extra compute buys you:** exploration, verification, or refinement.
- **Match the mechanism to the task:** automatic verifiers favor width; near-correct drafts favor depth.
- **Treat verification quality as a bottleneck:** bad verifiers can make stronger search worse.
- **Allocate compute adaptively:** question difficulty should affect the strategy.
- **Train for the inference protocol:** the model must learn how to exploit longer thinking or revision loops.

Takeaways: Test-Time Scaling

- Test-time scaling turns inference into a compute-allocation problem under a fixed budget.
- The two main axes are **width** (sample/search more) and **depth** (revise/think longer).
- Repeated sampling works when coverage improves and verification is reliable.
- Search and process verifiers help spend width more intelligently, especially on harder problems.
- Sequential revision and longer thinking budgets show that more depth can also buy accuracy.
- The strongest systems increasingly combine width and depth rather than choosing only one.

Next: If better reasoning needs good trajectories, how can training explicitly prefer stronger trajectories over weaker ones? This leads to GRPO.

Part III: Boosting Reasoning with GRPO

Part II asked how to spend more compute at inference time.

Part III asks the complementary question:

- How do we **train** a model to use that extra compute effectively?
- How do we move from imitating reasoning traces to **optimizing** for better ones?
- Why has **GRPO** become a central recipe in modern reasoning RL?

A Simpler Bridge: s1 Before RL

Before full RL, the s1 paper asks a useful bridge question:

Can strong reasoning behavior be elicited with **SFT plus inference-time control**?

Their recipe is intentionally simple:

- curate a small, high-quality reasoning dataset ($s1K$),
- perform supervised fine-tuning only,
- then use **budget forcing** to control thinking length at test time.

Budget forcing means:

- suppress stopping if the model ends too early,
- append <Wait> to keep the trace going,
- or terminate once a maximum budget is reached.

What s1 Tells Us Before RL

The main lesson from s1 is not that RL is unnecessary.

It is that some reasoning gains already come from the combination of:

- better reasoning demonstrations,
- a model trained to use longer traces,
- and explicit control over test-time compute.

Empirically, s1 also clarifies the same width–depth picture:

- **Sequential scaling:** spend more tokens on one trace via budget forcing.
- **Parallel scaling:** sample multiple traces and aggregate them.
- Both help, but they saturate differently and remain complementary.

This sets up Part III: once SFT has taught the model a reasoning style, RL asks how to *optimize beyond imitation*.

Why Move from SFT to RL?

Supervised fine-tuning (SFT) is the natural first step:

- collect target trajectories,
- maximize their likelihood,
- teach the model a style of reasoning that works reasonably well.

But SFT has a built-in limit:

- it only says *“imitate this trajectory”*,
- not *“discover a better trajectory that earns higher reward”*.

For reasoning tasks, that gap matters because the best path may be:

- different from the demonstration,
- longer than the demonstration,
- or only discoverable through exploration.

$$J_{\text{SFT}}(\theta) = \mathbb{E}_{(q,o) \sim P_{\text{SFT}}} \left[\frac{1}{|o|} \sum_{t=1}^{|o|} \log \pi_{\theta}(o_t \mid q, o_{<t}) \right]$$

SFT gives every target token the same positive coefficient; it does not compare alternative trajectories by reward.

SFT and RL Optimize Different Things

A useful way to frame the difference is by asking three questions.

What data do we learn from?

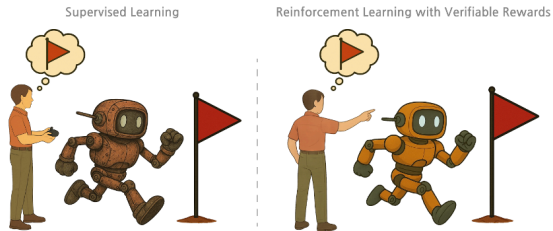
- SFT learns from a fixed dataset of demonstrations.
- RL learns from model-generated trajectories plus rewards.

What signal do we optimize?

- SFT treats every target token as equally correct supervision.
- RL weights updates by how good the sampled outcome was.

What capability does this encourage?

- SFT encourages imitation and formatting.
- RL encourages search, correction, and reward-seeking behavior.



The Policy Gradient Idea

Reinforcement learning starts from a simple objective:

maximize the expected reward of trajectories sampled from the current policy

Policy gradients operationalize this by:

- sampling trajectories from the model,
- evaluating how good they are,
- increasing the probability of actions that appear in high-reward trajectories,
- decreasing the probability of actions that appear in low-reward trajectories.

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)], \quad \nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau) \nabla_{\theta} \log \pi_{\theta}(\tau)]$$

$$\pi_{\theta}(\tau) = \prod_t \pi_{\theta}(o_t \mid q, o_{<t})$$

In LLM terms: a reasoning trace is a trajectory, and token probabilities are the policy.

Why Vanilla Policy Gradients Are Hard to Use

The clean idea becomes messy in practice.

Vanilla REINFORCE-style updates suffer from:

- **high variance:** one sampled trajectory can be extremely noisy,
- **sparse rewards:** reasoning tasks often provide feedback only at the end,
- **instability:** large updates can move the policy too far in one step.

This is why practical RL adds stabilizers such as:

- baselines and advantages,
- clipped updates,
- KL control against a reference policy.

From REINFORCE to PPO

PPO is easier to understand if we add stabilizers one by one rather than jumping straight to the final objective.

Start from REINFORCE:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau) \nabla_{\theta} \log \pi_{\theta}(\tau)] .$$

This says: increase the probability of trajectories with high reward.

But in practice that update is too noisy, so modern policy optimization adds:

- an **advantage** instead of raw reward,
- importance weighting by an **old policy**,
- clipping and KL control to keep updates local.

Advantage and the Value Function

The first fix is to replace raw return with an **advantage**:

$$A_t = Q(s_t, a_t) - V(s_t).$$

Intuition:

- $Q(s_t, a_t)$: how good was the sampled continuation?
- $V(s_t)$: how good was this state on average?
- A_t : was this action better or worse than expected?

So the update becomes:

$$\nabla_{\theta} J(\theta) \approx \mathbb{E} [A_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] .$$

Using A_t lowers variance: we no longer reward every sampled token equally, only tokens that look *better than baseline*.

In PPO, that baseline is usually estimated by a learned **value model** or critic.

Importance Sampling and the PPO Ratio

The next issue is practical: data were sampled from the **old policy**, but we want to update the **current policy**.

A concrete PPO-style example is:

- roll out 128 prompts with the old policy,
- split them into minibatches of 32 prompts,
- make 4 gradient updates by reusing those same sampled trajectories.

After the first update, the policy has already changed, so the remaining minibatches are slightly *off-policy*.

PPO corrects for that mismatch with the importance ratio

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$

This ratio tells us how much more or less probable the sampled action becomes under the updated policy.

The resulting surrogate objective is

$$\mathbb{E}[r_t(\theta)A_t].$$

If $r_t(\theta)$ gets too large or too small, the reused data are no longer trustworthy: the current policy has drifted too far from the rollout policy.

Clipping and KL Control in PPO

PPO keeps updates local by clipping the importance ratio:

$$L_{\text{PPO}}(\theta) = \mathbb{E} [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] .$$

Intuition:

- if the new policy stays close to the old one, optimize normally;
- if the ratio moves too far outside $[1 - \epsilon, 1 + \epsilon]$, stop trusting that update.

In RLHF-style LLM training, PPO is also usually paired with:

- a **reward model**,
- a **reference model**,
- and a KL penalty such as $-\beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$ to keep outputs near the SFT policy.

So PPO is really REINFORCE plus **advantage estimation**, **importance sampling**, **clipping**, and **KL regularization**.

Why PPO Is Awkward for Reasoning LLMs

DeepSeekMath emphasizes two practical issues with PPO in the LLM setting.

First: the critic is expensive.

- PPO usually trains a value model comparable in size to the policy.
- That adds substantial memory and compute cost.

Second: the critic is hard to fit for sparse reasoning rewards.

- many reasoning tasks only know whether the final answer is correct,
- but PPO wants token-level or step-level value estimates.

This creates exactly the opening that GRPO exploits: keep the policy optimization benefits, but avoid the critic.

GRPO: The Core Idea

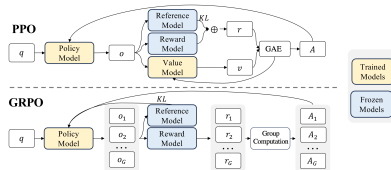
Group Relative Policy Optimization (GRPO) is a PPO-style algorithm with a simpler baseline.

For each prompt:

- sample a **group** of outputs from the old policy,
- score them with a reward function,
- compare each output *relative to the others in the same group*,
- update the policy using those relative advantages.

The key simplification is:

- no separate critic model is needed,
- the group reward statistics provide the baseline.



$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}$$

Here the advantage is computed *within the sampled group for the same prompt*, not from a learned value function.

From PPO to GRPO: What Changes?

Conceptually, PPO says:

- estimate an advantage with a learned value function,
- then update the policy using a clipped objective.

GRPO says:

- sample several outputs for the same question,
- compute rewards for each,
- normalize rewards within the group,
- use that normalized relative score as the advantage.

So the shift is not from policy gradients to something else.

It is from **critic-based advantage estimation** to **group-relative advantage estimation**.

$$L_{\text{GRPO}}(\theta) = \mathbb{E}_{q, \{o_i\}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_t \min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) - \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}}) \right]$$

where $r_{i,t}(\theta) = \pi_\theta / \pi_{\theta_{\text{old}}}$ at token t .

Why Group-Relative Advantages Make Sense

The design of GRPO matches reasoning tasks surprisingly well.

- Reward models are often trained comparatively: which answer is better for the *same* prompt?
- Verifiable tasks are naturally relative: among several attempts, some are correct and others are not.
- Group normalization reduces sensitivity to absolute reward scale and focuses the update on *which trajectory was better*.

This is why GRPO feels especially natural for:

- math,
- code,
- logic,
- and other tasks with prompt-specific success criteria.

Why GRPO Became Attractive in Practice

Relative to PPO, GRPO offers a pragmatic package:

- lower memory usage because it removes the value model,
- simpler training dynamics for sparse end-of-trajectory rewards,
- direct compatibility with prompt-level comparative evaluation,
- strong empirical performance on mathematical reasoning.

DeepSeekMath uses GRPO to push a 7B model far beyond its SFT stage, showing that RL can meaningfully upgrade reasoning even after good instruction tuning.

RLVR: Reinforcement Learning with Verifiable Rewards

GRPO becomes especially powerful in the RLVR regime.

RLVR means:

- the model generates candidate reasoning traces,
- a rule-based verifier or exact checker assigns reward,
- RL updates the policy toward trajectories that solve the task.

Intuitively, the reward can be as simple as

$$r(q, o) = \mathbf{1}[\text{Verify}(q, o) = \text{pass}] ,$$

where the verifier checks whether the final answer, proof, or program is correct.

This is attractive because it avoids many ambiguities of preference modeling:

- math answers can be checked,
- code can be compiled and tested,
- logical tasks can often be verified symbolically,
- and it is usually harder to hack than a learned reward model because success must satisfy an external checker, not merely exploit reward-model blind spots.

R1-Zero: Pure RL Can Elicit Reasoning

DeepSeek-R1-Zero is the cleanest demonstration of RLVR with GRPO.

The recipe is strikingly simple:

- start from a base model,
- skip the usual reasoning SFT stage,
- train with GRPO using rule-based rewards on math, code, and logic tasks.

Rewards are primarily:

- **accuracy rewards** for correct answers,
- **format rewards** for producing explicit `<think>` and `<answer>` structure.

$$R_{\text{rule}} = R_{\text{acc}} + R_{\text{format}}$$

The key claim is not just higher accuracy, but the emergence of reasoning behaviors during RL.

What Emerges in R1-Zero

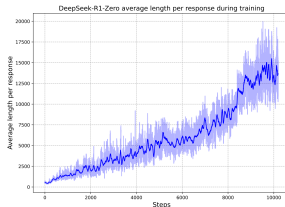
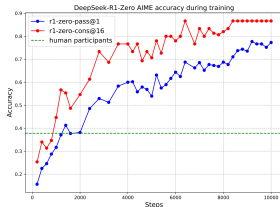
The R1 paper reports several emergent behaviors during GRPO training:

- longer reasoning traces,
- self-reflection and backtracking,
- verification and re-checking,
- dynamic adaptation of strategy.

Two especially memorable observations are:

- response length increases sharply during training,
- the model exhibits an “aha moment” where it explicitly rethinks its earlier path.

This is one of the strongest pieces of evidence that RL can *induce* reasoning behavior, not merely copy it.



Why R1 Does Not Stop at Pure RL

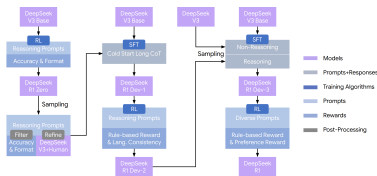
R1-Zero shows that RL can *induce* strong reasoning, but pure RL is not yet the full recipe.

The paper highlights three practical issues:

- poor readability,
- mixed languages inside one chain of thought,
- weak coverage of general non-reasoning behavior.

The figure shows a hybrid pipeline rather than “just do RL longer”:

- 1. V3-Base → RL on reasoning prompts with accuracy and format rewards → R1-Zero.
- 2. Sample R1-Zero traces, then **filter** and **refine** them into cold-start long-CoT data; SFT from V3-Base gives R1-Dev-1.
- 3. R1-Dev-1 → RL on reasoning prompts with rule-based reward and language consistency → R1-Dev-2.
- 4. Sample reasoning data from R1-Dev-2, mix it with non-reasoning data from DeepSeek-V3, then run another SFT from V3-Base to get R1-Dev-3.
- 5. R1-Dev-3 → final RL on diverse prompts with rule-based and preference rewards → DeepSeek-R1.



The Main Lesson from R1

R1 suggests a practical division of labor between SFT and RL.

SFT contributes

- formatting,
- readability,
- conversational alignment,
- and a reasonable starting policy.

RL contributes

- exploration beyond demonstrations,
- stronger verifiable-task performance,
- longer and more adaptive reasoning behavior.

In other words: SFT teaches the model *how to speak*; RL teaches it to *search for better solutions*.

The Limitation of On-Policy RLVR

Standard GRPO-style RLVR is still mostly **on-policy**.

So the model learns mainly from its own sampled trajectories.

This creates two bottlenecks:

- **Weak-model collapse:** if the current policy is weak, most sampled trajectories are poor, so reward and advantage signals become too weak to drive improvement.
- **Hard-problem barrier:** on problems the model can barely solve at all, it may never visit the useful intermediate states needed for RL to bootstrap.

In short, the exploration horizon is bounded by what the model can already generate.

This motivates the next question: **how do we inject external guidance without giving up RL?**

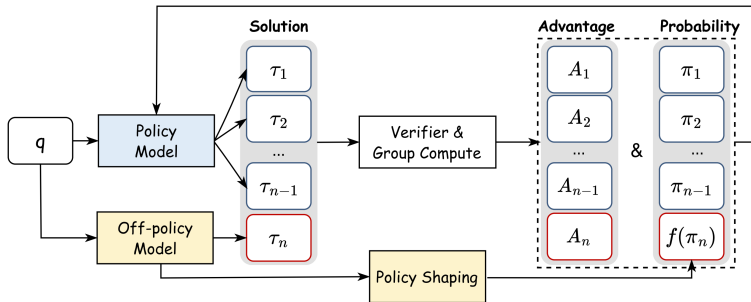
LUFFY: Off-Policy GRPO

LUFFY answers this by injecting **off-policy reasoning traces** into the GRPO update.

Read the figure from left to right:

- the policy model still generates the main group of candidate solutions,
- an off-policy model contributes an external high-quality trace,
- verifier and group computation assign relative advantages over the mixed group,
- policy shaping modifies the off-policy probability term so guidance helps without naive copying.

The point is not to replace RL with imitation, but to *expand the search horizon* when the current policy cannot discover good trajectories alone.



Why Off-Policy Guidance Helps

Mixed-policy GRPO changes the learning dynamics:

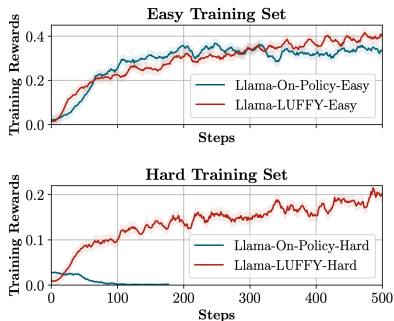
- when the model is weak, high-quality off-policy traces dominate the relative advantage signal,
- as the model improves, successful on-policy rollouts begin to take over,
- the training naturally shifts from imitation toward self-driven exploration.

This is a useful compromise:

- not pure SFT, which only imitates,
- not pure on-policy RL, which may never find a good path,
- but an adaptive combination of both.

What LUFFY Shows

LUFFY matters not only because it improves average performance, but because it can still train in regimes where on-policy RLVR fails.



This figure uses LLaMA-3.1-8B on two subsets of different difficulty:

- on the **easy** subset, both on-policy RLVR and LUFFY improve over training;
- on the **hard** subset, on-policy RLVR collapses to near-zero reward, because its own rollouts rarely obtain positive feedback;
- LUFFY still improves stably by injecting strong off-policy traces.

The takeaway is robustness: off-policy guidance helps learning go beyond the current model's exploration horizon.

Takeaways: GRPO and Beyond

- SFT and RL do different jobs: SFT gives a readable, aligned starting policy; RL improves decisions under task reward.
- GRPO makes RLVR practical for reasoning by replacing the critic with within-group relative advantages, especially when rewards are verifiable.
- R1 and LUFFY suggest the practical recipe is not “pure RL”: combine RL with good initialization, curated data, and, when needed, external off-policy guidance.

Overall message: test-time scaling spends more compute at inference; GRPO-style training teaches the model to use that compute more effectively.

Final Takeaways

- Reasoning matters because many tasks cannot be solved reliably by one-step pattern matching; they need decomposition, verification, and adaptive computation.
- Test-time scaling treats inference as a compute-allocation problem: spend more budget on hard problems through width, depth, or both.
- GRPO and its extensions treat training as a policy-improvement problem: teach the model to produce stronger reasoning trajectories before inference even begins.
- The practical frontier is therefore *jointly* optimizing training-time reasoning ability and test-time reasoning budget, rather than relying on either one alone.

Reason better, spend compute better, and align the two.